SERVER SCRIPT

CREATE_PRODUCTION_PLAN_FROM_PLANNING_SHEET

Script Type: API

Event Frequency: All

DocType Event: Before Insert

API Method: create_production_plan_from_planning_sheet

Script

```
# SCRIPT: SYNC PRODUCTION PLAN - FINAL VERSION
# TYPE: API
# METHOD NAME: create_production_plan_from_planning_sheet

planning_sheet = frappe.form_dict.get("planning_sheet")
if not planning_sheet:
    frappe.throw("Planning Sheet not provided")

ps = frappe.get_doc("Planning sheet", planning_sheet)

party_code = ps.party_code or ""
sales_order = ps.sales_order

company = frappe.db.get_default("company") or frappe.db.get_value("Global Defaults", None, "default_company")
if not company:
    frappe.throw("Default Company not set")

today = frappe.utils.nowdate()

# ========== GROUPING: Unit + Quality + Color ==========
group_map = {}

# Filter items: Must have Unit assigned and be Active (docstatus check not needed if fetching from child table direc
tly, but good practice)
for r in ps.items:
    if not r.unit:
        continue

    # Normalize values for grouping
    unit = str(r.unit).strip()
    quality = (r.custom_quality or "").strip().upper() or "NO_QUALITY"
    color = (r.color or "").strip().upper() or "NO_COLOR"

    # Unique Key
    key = f"{unit}||{quality}||{color}"

    group_map.setdefault(key, []).append(r)
```

```
if not group_map:
    frappe.throw("No valid Unit/Items found in Planning Sheet")

# Helper to get BOM
def get_bom(item_code):
    bom = frappe.db.get_value("BOM", {"item": item_code, "is_active": 1, "is_default": 1}, "name")
    if not bom:
        bom = frappe.db.get_value("BOM", {"item": item_code, "is_active": 1}, "name")
    return bom

# ========== FETCH EXISTING PRODUCTION PLANS ==========
existing_pp = frappe.get_all("Production Plan",
    filters={"custom_planning_sheet": ps.name, "docstatus": 0},
    fields=["name", "custom_unit", "custom_quality", "custom_color"]
)

existing_map = {}
for pp in existing_pp:
    ex_unit = (pp.custom_unit or "").strip()
    ex_quality = (pp.custom_quality or "").strip().upper() or "NO_QUALITY"
    ex_color = (pp.custom_color or "").strip().upper() or "NO_COLOR"
    ex_key = f"{ex_unit}||{ex_quality}||{ex_color}"
    existing_map[ex_key] = pp.name

created = []
updated = []

# ========== CREATE/UPDATE PRODUCTION PLANS ==========
for key, rows in group_map.items():
    parts = key.split("||")
    unit_val = parts[0]
    quality_val = parts[1] if parts[1] != "NO_QUALITY" else ""
    color_val = parts[2] if parts[2] != "NO_COLOR" else ""

    if key in existing_map:
        # UPDATE
        pp = frappe.get_doc("Production Plan", existing_map[key])
        pp.set("po_items", []) # Clear items to rebuild
        updated.append(pp.name)
        is_new = False
    else:
        # CREATE
        pp = frappe.new_doc("Production Plan")
        pp.company = company
        pp.posting_date = today
        pp.sales_order = sales_order
```

```
        pp.custom_planning_sheet = ps.name
        pp.custom_party_code = party_code
        pp.custom_unit = unit_val
        pp.custom_quality = quality_val
        pp.custom_color = color_val
        created.append("NEW")
        is_new = True

    # Common Flags
    pp.flags.ignore_permissions = True
    pp.flags.ignore_mandatory = True

    # Add Items
    for r in rows:
        planned_qty = float(r.qty or 0)
        if planned_qty <= 0: continue

        bom = get_bom(r.item_code)
        item_uom = frappe.db.get_value("Item", r.item_code, "stock_uom") or "Kg"

        pp.append("po_items", {
            "item_code": r.item_code,
            "bom_no": bom,
            "planned_qty": planned_qty,
            "uom": item_uom,
            "stock_uom": item_uom,
            "conversion_factor": 1,
            "sales_order": sales_order,
            "sales_order_item": r.sales_order_item,
            "description": r.description or r.item_name,
            # Custom Fields (Traceability)
            "custom_party_code": party_code,
            "custom_unit": unit_val,
            "custom_quality": quality_val,
            "custom_color": r.color or "",
            "custom_planning_sheet": ps.name,
            "custom_gsm": float(r.gsm or 0),
            "custom_width_": float(r.width_inch or 0),
            "custom_meterperroll": float(r.meter_per_roll or 0),
            "custom_weight_per_roll": float(r.weight_per_roll or 0),

            # ▒ ADDED: Fetching 'no_of_rolls' from Planning Sheet Item to 'custom_no_of_rolls'
            "custom_no_of_rolls": int(r.no_of_rolls or 0),

            # Split traceability (if fields exist in PP Item)
            # "custom_split_from": r.custom_split_from
```

```
        })

    if is_new:
        pp.insert(ignore_permissions=True)
        if created and created[-1] == "NEW":
            created.pop()
            created.append(pp.name)
        existing_map[key] = pp.name # Track just created
    else:
        pp.save(ignore_permissions=True)

# ========== CLEANUP ORPHANS ==========
for pp_key, pp_name in existing_map.items():
    if pp_key not in group_map:
        old_pp = frappe.get_doc("Production Plan", pp_name)
        if old_pp.docstatus == 0:
            old_pp.delete(ignore_permissions=True)

frappe.db.commit()

frappe.response["message"] = {
    "success": True,
    "created": created,
    "updated": updated
}
```

Request Limit:	5
Time Window (Seconds):	86400